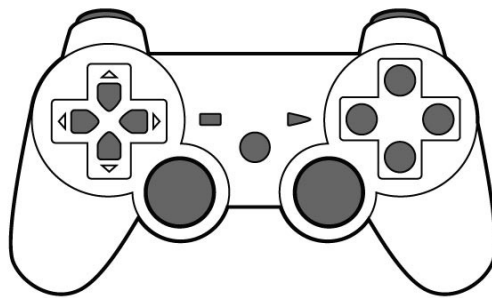


Exploring the Structure and Function of NPC Dialogue Through Machine Learning



Group Nine Logbook:

Emma Clark,
Scott Morrison,
Leah Colwell,
Stephen Kelly
& Ruairidh Williams

Contents:

Content:.....	1
Colab link:.....	3
The Idea:.....	3
The Project:.....	3
The Brainstorming:.....	4
The Process:.....	4
Preparing the data for AI Processing.....	5
Leah - Ethical Research:.....	6
Stephen - Fallout.....	7
Scott - People of Skyrim.....	11
Emma - Exploratory Testing of Pre-trained Dialogue Models.....	11
Ruaridh - Knights of the Old Republic.....	11

Overview:

Colab link:

<https://colab.research.google.com/drive/1Fqgx0Z7ceeeUwtrQw-ut0NJNriAEL4iN?usp=sharing>

The Idea:

Why NPC Dialogue Matters

In video games, non-player characters (NPCs) play a crucial role by contributing to narrative depth, world-building and player immersion. The bulk of this is done through dialogue; by interacting with the player, NPCs can provide information, express personality, and unlock achievements, all while helping create the illusion of a ‘real world’. These NPCs have evolved over the years; their dialogue was traditionally created using static scripts/dialogue trees, which often result in repetitive, predictable interactions—limiting their ability to respond dynamically to the player and overall reducing immersion.

Enter Machine Learning

Nowadays, we can use machine learning and natural language processing to create more adaptive and responsive dialogue systems. Machine learning models are efficient at identifying patterns in large text datasets, making them well-suited for analysing the structure, tone, and recurring themes in NPC dialogue. By learning data from existing dialogue, ML models can generate responses that more closely resemble human-like conversations.

The Project:

As a group, we explored the structure and function of NPC dialogue through machine learning by training chatbot models on character-specific dialogue datasets. Using Google Colab as a development environment, dialogue from individual NPCs was used to train models designed to replicate distinct speaking styles and personality traits. The resulting chatbots were then interacted with to evaluate how effectively the models captured each NPC's unique characteristics. We aimed to investigate whether machine learning could simulate NPC personalities solely from dialogue, enabling us to examine how language patterns contribute to the perception of character identity. By demonstrating how machine learning can generate personality-driven dialogue, this research highlights the potential for more dynamic NPC interactions and enhanced player immersion in future game design.

The Brainstorming:

Working out the Title

As a group, we decided to explore NPCs' dialogue first. We had the title 'How does AI create NPC dialogue-based story context and task level?' However, after discussing everyone's skill level and the project's time frame, we settled on 'Exploring the Structure and Function of NPC Dialogue Through Machine Learning' so we could focus on how the AI decides which dialogue option to use rather than creating one from scratch.

Establishing a Practical Objective

Our focus was input → AI processing → output. Our AI system would take an input: a collection of dialogue input from different existing NPCs, and from that, it would output a context-aware, in-character response that varies in tone, complexity, and content based on these factors. One of our worries, however, was whether this was measurable; could we generate examples and compare variations?

The Process:

Getting Familiar with Google Colab

At our first meeting, the group set up and explored Google Colab through a shared notebook so that everyone could access and edit Python code. We then discussed the logistics of how we would approach the project: possible datasets, how the dialogue would need to vary based on story context/player task/NPC role, etc., and whether accents or speech patterns could be inferred. We also mentioned the ethical challenges that we could face and identified that we would need to consult relevant academic literature to support later analysis. At the end, we determined that the session helped clarify the project direction and confirmed that NPC dialogue generation is a suitable and interesting application of AI. Further research is needed to identify appropriate datasets and define a feasible scope.

Specifying/Finding the data

Next, we needed to determine which data type would make the project work. We found that the most practical was mostly textual/symbolic (natural language dialogue), potentially mixed if you add metadata (e.g., NPC role as categorical, task level as ordinal). From this, we came up with a list of requirements:

1. Dialogue examples that show variation (branching, context-dependent responses).
2. Labelled or structured where possible (e.g., speaker context tags, quest stage).
3. Why? To prompt/fine-tune an LLM or analyse patterns (e.g., how dialogue shortens/simplifies for low-level tasks).
4. Volume: Must have 1-2 datasets (hundreds to thousands of text lines) - enough for demos and analysis without overwhelming Colab limits.

Once we knew the criteria, we began looking at various public datasets online. We collected a wide range of possible datasets (most were downloadable in Colab via Wget, Github raw links, or Hugging Face datasets library):

- hmi-utwente/video-game-text-corpora
- <https://github.com/seannyD/VideoGameDialogueCorpusPublic>
- <https://huggingface.co/datasets/amaydler/npc-dialogue>
- <https://www.kaggle.com/datasets/muhajipra/people-of-skyrim>
- <https://github.com/jd7h/sentiment-lexicon-skyrim/tree/master>
- <https://github.com/art-from-the-machine/Mantella-LLM-Fine-Tuning?tab=MIT-1-ov-file>
- <https://art-from-the-machine.github.io/Mantella/>
- https://fallout.fandom.com/wiki/Category:Fallout_dialogue_files
- <https://correlation-machine.com/VideoGameDialogueCorpus/BG3Explorer/>
- <https://dillonbecker.itch.io/sdap>

While we did not use all of these in our project, it demonstrates that we all attempted to find suitable datasets.

Preparing the data for AI Processing

However, we did consider some possible issues and worked on how we could solve them:

1. Some datasets often have incomplete branches of text (missing values); the solution was to use complete subsets to fill in those gaps with LLM generation.
2. The possible data types (numerical/symbolic/mixed). In our case, we needed purely symbolic/text; we would only need numeric for analysis.

We found that these problems, while possible, did not occur as often as we would have expected when beginning to explore the datasets.

(PLEASE NOTE: We all originally kept our own little logbooks with different areas of research, different prototypes, etc., which were then combined into this document. Multiple datasets were explored by different members before the final was settled on.)

Leah - Ethical Research:

The Audience:

We found that the ethical considerations of this project were closely tied to the growing influence of video games as a source of language exposure and cultural representation. Rennick (Stephaine Rennick, 2021)¹ highlights that video game dialogue is becoming increasingly relevant as linguistic input, particularly for younger audiences who spend significant time engaging with games and, as such, have widespread exposure to NPCs and their dialogues. She suggests that children are spending around an hour a day playing video games. (Rennick, 2021) As such, using NPC dialogue as training data for machine learning systems raises ethical questions about how language, attitudes, and character traits are reproduced and potentially amplified by AI models, and what exposure to these characteristics could entail.

The Law:

It is also crucial to consider the methods which we used to obtain the dialogue data. They must necessitate careful consideration of copyright, consent and appropriate use; this is essential, particularly when dialogue is being repurposed for research and AI training. Rennick lists four ways in which it is possible to obtain dialogue from video games (Rennick, 2021)

1. Extracting dialogue directly from game data.
2. Manual transcription of narrative permutations
3. Using fan transcripts.
4. Using wikis and community websites.

However, certain games, such as Fallout New Vegas, have released an official modding tool called the Garden of Eden Creation Kit that allows players to extract all in-game dialogue. (Hamalainen, 2022)²

Emotional Implications:

¹ Hämäläinen, M., Alnajjar, K and Poibeau, T. (2022). Video Games as a Corpus: Sentiment Analysis using Fallout New Vegas Dialog. *catalyzex.com*. [online] [doi:https://eprints.gla.ac.uk/291596/2/291596.pdf](https://eprints.gla.ac.uk/291596/2/291596.pdf).

² Callison-Burch, C., Tomar, G.S., Martin, L.J., Ippolito, D., Bailis, S. and Reitter, D. (2022). Dungeons and Dragons as a Dialog Challenge for Artificial Intelligence. arXiv:2210.07109 [cs]. [online] Available at: <https://arxiv.org/abs/2210.07109>.

Ethical concerns also arise regarding not just language but also the emotional representation and character portrayal. Research on sentiment analysis in video game dialogue demonstrates that NPC speech often conveys nuanced emotional states such as anger, fear, or sadness, which are central to character identity and narrative meaning. (Hamalainen, 2022) If machine learning models are trained on such data without critical oversight, there is a risk of misrepresenting emotional tone or reinforcing biased or stereotypical portrayals. Also, inconsistencies across language dubs suggest that emotional nuance can be lost or distorted, leading to uncertainty about AI systems' responsibility to preserve the intended tone and meaning when generating dialogue.

Other AI Dialogue Systems:

There have been numerous studies conducted in role-playing environments, such as Dungeons & Dragons, to evaluate AI dialogue systems, illustrating the importance of control and transparency in AI-generated interactions. They use conditioning signals or control features to guide dialogue generation, highlighting the ethical need for developers to maintain oversight over how AI systems behave and the roles they assume. The article overall emphasises the importance of AI as an assistant rather than as a player.

'The LM can respond as a particular character or as the player who runs the game—i.e., the Dungeon Master (DM). It is trained to produce dialogue that is either in-character (roleplaying in the fictional world) or out-of-character (discussing rules or strategy). We perform a human evaluation to determine what factors make the generated output plausible and interesting. We further perform an automatic evaluation to determine how well the model can predict the game state given the history and examine how well tracking the game state improves its ability to produce plausible conversational output.' (Callison-Burch, 2022)³

In the context of our NPC chatbots, we need to ensure that generated dialogue remains appropriate, context-aware, and aligned with the character's intended personality is essential to prevent misleading or harmful interactions.

³ Rennick, S. and Roberts, S. (2024). The Video Game Dialogue Corpus. *Gla.ac.uk*. [online] doi:<https://eprints.gla.ac.uk/291596/2/291596.pdf>.

Stephen – Fallout

Week 1:

During week 1 we got used to using Colab and experimenting with different prompts to see what it does, how it works and how successful Gemini is with working to the prompts provided.

The initial thoughts on it were positive, Gemini threw out working code faster than expected, the prompts weren't complex or intricate, but it was still interesting to see how well it worked with the initial prompts.

We decided to try some more complex prompts and found out that it required a lot more precise prompts and phrasing for the instructions given to Gemini to have the same sort of success results. Gemini would sometimes struggle to keep the context of the instructions when given another prompt and opening a new Colab cell. From this we learned that we needed to reiterate relevant information from past Colab cells within the new prompts. In the definition of trial-and-error working, it was good to see Gemini creating programs and syntaxes quickly, but it requires a human overseer to keep it on track, becoming more critical reviewing, each result it gave.

This week was informative on how good of an accelerator AI can be to a software engineer/ developer when used as a tool.

Week 2:

Transition from simple experimentation of Colab and feeding Gemini the initial concept of creating a NPC which is driven by provided datasets. Gemini used the datasets to create and simulate an NPC persona. There was however a clear issue with Gemini grasping the concept as the execution needed attention as it did not work as intended. There were errors with the code running which were again, fixed by Gemini.

The main issue we encountered was that there was no way to interact with the NPC as there was no input box or option for the user to use. So, at the starting line of this program the NPC was essentially mute after its first output of telling the user who it was. This needed to be fixed.

There was a lot of troubleshooting that had to be done, feeding the prompts of, "Please fix the errors", which Gemini then debugged for me such as making sure the datasets were formatted correctly. After the focus changed to the features that the program would have, specifically adding a loop so that the user could converse with the NPC via the inclusion of a input box. trying to include this caused further problems and breakages such as conversation history not being retained or the input was not being answered correctly by the NPC. The need for consistent re-prompting while isolating each issue that came up was tiresome and repetitive. This was eventually resolved after a few hours of working with Gemini and a lot of prompts starting with "No that is not what I

wanted". All of this was interesting and informative in the learning process for building interactive features with the help of AI, knowing now that there will be a lot of issues with code breaking and debugging needing to be done before anything will work with more complex programs. There is no "one shot" solution which we found out as the input box still does not work.

Week 3:

The main objective for this week was to get the two-way communication between the NPC and the user working and comprehensive enough for the user to realise there is an option to with the inclusion of an input box. Making the input box a consistent part of the program was important as it kept disappearing before. This was fixed and then we also wanted to include microphone and voice detection of the user to try replicate a natural vocal conversation. There was again a lot of trial and error with this as loops and audio input from the microphone kept breaking and still does not work without a dedicated microphone in the device.

The consistent inclusion of the input box came with usability challenges but it is disappearing after the first response from the NPC essentially forcing the user to restart the cell every time they want to talk to the NPC but then when doing that there is not option to test the retention of the NPC from what has been said before.

Besides this issue the NPC can now communicate through text and audio responses which is showing progress. The quality of the NPC's responses is still inconsistent as the output is always incomplete sentences which cut off halfway through which we think may be a token amount usage limit that Gemini has included in the code.

Week 4:

This week was mainly dedicated to upgrading the NPC's language capabilities and stability due to some initial failures of the code running. There was a focus on getting rid of the cut off sentences which was done by increasing the token amount in the code from 50 to 100 and then some prompting to have the program avoid cutting off mid-sentence, so the NPC now talks to the user in complete and understandable sentences.

After, there was a new dataset added to give the program more to choose from when speaking to the user at a base level, such as more responses that could fit the input from the user. This however brought on issues with the panada library with compatibility errors, which was quickly fixed.

We then attempted to add AI to the NPC and making sure it stuck to the context of the datasets but still allowing it to come up with responses as well as responding with dialogue from the datasets as well. This threw multiple syntax errors which was related to "vector embeddings". Through troubleshooting,

research on what to prompt and then prompting Gemini in Colab the program was able to fix the embedding logic by stabilising it and this ensured the NPCs reliability to access the background information of the datasets without the session crashing or having to be restarted.

After sorting out the issues with the foundations of the program the attention was turned to the “behaviour”, making the AI NPC more contextually aware through refining the prompts used and encouraging it to proactively drive conversation with the user through getting the NPC to ask questions back to the user that were relevant rather than it just passively answering questions proposed by the user. By doing this it led to an interesting, but unintended, effect in which the model then began making up entirely new characters in its own narrative, which would also ask questions and respond to the users as well. If there was more time to work on this project, we would have been keen to explore this but for now we had Gemini remove this with strict negative prompting to constrain the AI NPC to solely its own persona.

While the NPC did repeat itself sometimes the overall interaction and flow of conversation improved greatly and began being an actual conversational NPC.

Week 5

We still had to fix some of the repetition as the more we talked with the AI NPC the worse the repetition got from it, to curb this there was the inclusion of more negative prompts for repetition which sorted it out, this also brought back the issue of the AI NPC doing more narration of the user which was also prompted out of the program by giving constrictions and negative prompts to colab.

```
import pandas as pd
from datasets import load_dataset
import random
import sys
import torch
import numpy as np
import os

# --- Installation Block ---
try:
    import speech_recognition as sr
    from gtts import gTTS
    from sentence_transformers import SentenceTransformer, util
except ImportError:
    print("Installing dependencies...")
    !pip install SpeechRecognition gTTS sentence-transformers datasets > /dev/null 2>&1
    !apt-get update > /dev/null 2>&1
    !apt-get install -y portaudio19-dev > /dev/null 2>&1
    !pip install PyAudio > /dev/null 2>&1

import speech_recognition as sr
from gtts import gTTS
from sentence_transformers import SentenceTransformer, util

import IPython.display as ipd
from transformers import pipeline

# --- 1. Load Datasets & Build "World Knowledge" ---
print("Initializing the Wasteland... (Loading Datasets)")

# A. Load NPC Dialogue (The "Personalities")
ds_npc = load_dataset("amaydle/npc-dialogue")
df_npc = pd.DataFrame(ds_npc['train'])
```

```

# Indexing the Knowledge Base
if knowledge_base:
    print("Indexing Lore...")
    knowledge_embeddings = semantic_model.encode(knowledge_base, convert_to_tensor=True)
else:
    knowledge_embeddings = None

# Indexing the Voice Bank (for exact line matching)
if voice_bank:
    print("Indexing Voice Lines...")
    voice_texts = [v['text'] for v in voice_bank]
    voice_embeddings = semantic_model.encode(voice_texts, convert_to_tensor=True)
else:
    voice_embeddings = None

# --- Helper: Trim to Complete Sentence ---
def clean_sentence(text):
    """Ensures the NPC doesn't trail off into silence."""
    last_punct = max(text.rfind('.'), text.rfind('!'), text.rfind('?'))
    if last_punct != -1:
        return text[:last_punct+1]
    return text

# --- 3. Random Identity & Motivation Generator ---
def generate_random_identity():
    """
    Creates a unique character with a specific MOTIVATION.
    This motivation drives them to ask questions.
    """
    factions = [
        "The Brotherhood of Steel", "The Railroad", "The Institute",
        "The Minutemen", "Diamond City Security", "Goodneighbor Drifter",
        "Wasteland Scavenger", "Raider Gang"
    ]

    personalities = [
        "paranoid and twitchy", "stoic and disciplined", "cheerful but shady",
        "weary and cynical", "aggressive and rude", "intellectual and arrogant"
    ]

    names = [
        "Jack", "Sarah", "Stitch", "Paladin Vance", "Scrapper Joe",
        "Dr. Zimmer", "Desdemona's Agent", "Trashcan Carla's Cousin", "Unknown Survivor"
    ]

    # New: Motivations ensure they have a reason to ask questions
    motivations = [
        "wants to know if the stranger has any caps or trade goods",
        "needs to find out if the stranger is a Synth spy",
        "is looking for news about a nearby settlement",
        "is lonely and just wants to hear a story",
        "is assessing if the stranger is a threat to the faction",
        "is looking for a missing person"
    ]

    faction = random.choice(factions)
    personality = random.choice(personalities)
    name = random.choice(names)
    goal = random.choice(motivations)

    return name, faction, personality, goal

# --- 4. The Inquisitive NPC Class ---
class InquisitiveNPC:
    def __init__(self, knowledge_embeds, knowledge_texts, voice_bank, voice_embeds):
        # Generate Identity + Motivation
        self.name, self.faction, self.personality, self.goal = generate_random_identity()

        self.knowledge_embeds = knowledge_embeds

```

```
self.knowledge_texts = knowledge_texts
self.voice_bank = voice_bank
self.voice_embeddings = voice_embeddings

def _recall_memory(self, player_query):
    """Searches the knowledge base for context/lore."""
    if self.knowledge_embeddings is None: return None

    query_embed = semantic_model.encode(player_query, convert_to_tensor=True)
    cos_scores = util.cos_sim(query_embed, self.knowledge_embeddings)[0]
    best_idx = torch.argmax(cos_scores).item()

    if torch.max(cos_scores).item() > 0.3:
        return self.knowledge_texts[best_idx]
    return None

def _check_voice_match(self, response_text):
    """Checks if the generated response matches a pre-recorded line."""
    if self.voice_embeddings is None: return None

    resp_embed = semantic_model.encode(response_text, convert_to_tensor=True)
    cos_scores = util.cos_sim(resp_embed, self.voice_embeddings)[0]
    best_idx = torch.argmax(cos_scores).item()

    # High threshold: Only use real audio if it fits perfectly
    if torch.max(cos_scores).item() > 0.70:
        return self.voice_bank[best_idx]
    return None

def interact(self, player_input):
    # 1. Context Retrieval
    memory = self._recall_memory(player_input)

# B. Load Fallout 4 Voices (The "Lore" & "Voice")
try:
    # Streaming mode avoids downloading the massive full dataset
    voice_ds_stream = load_dataset("AquaV/fallout-4-voices", split="train", streaming=True)

    knowledge_base = [] # Stores lore/facts
    voice_bank = [] # Stores audio files for matching

    print("Scavenging audio logs and holotapes...")
    # We load 300 lines to give the NPC a decent "memory" of the world
    for i, item in enumerate(voice_ds_stream.take(300)):
        text = item.get('text') or item.get('sentence') or item.get('transcription')
        if text:
            knowledge_base.append(text)
            voice_bank.append({
                'text': text,
                'audio': item['audio']['array'],
                'sampling_rate': item['audio']['sampling_rate']
            })

except Exception as e:
    print(f"Warning: Radio silence. Fallout dataset failed ({e}).")
    knowledge_base = ["War... war never changes."]
    voice_bank = []

# --- 2. Initialize AI Neural Nets ---
print("Bootting up RobCo Terminals...")

# Text Generation (The Brain)
generator = pipeline('text-generation', model='gpt2', pad_token_id=50256)

# Semantic Understanding (The Conscience)
semantic_model = SentenceTransformer('all-MiniLM-L6-v2')
```

```

# 2. Construct the Prompt with INQUISITIVE Instructions
context_str = f"World Context: '{memory}'" if memory else "World Context: No specific lore."

prompt = (
    f"You are roleplaying a character in Fallout.\n"
    f"Name: {self.name}\n"
    f"Faction: {self.faction}\n"
    f"Personality: {self.personality}\n"
    f"Current Goal: {self.goal}\n"
    f"---\n"
    f"{context_str}\n"
    f"---\n"
    f"Instruction: Respond *only* as the NPC character. Absolutely DO NOT invent or include dialogue\n"
    f"Traveler: {player_input}\n"
    f"{self.name}:"
)

# 3. Generate Response (Increased tokens slightly to allow for questions)
ai_output = generator(
    prompt,
    max_new_tokens=200, # Keep higher for complete thoughts/questions
    num_return_sequences=1,
    no_repeat_ngram_size=3, # Penalize repeating 3-word sequences
    temperature=0.7 # Slightly reduce for less creative deviation
)
raw_text = ai_output[0]['generated_text']

# Extract dialogue
response_text = raw_text.split(f"{self.name}:")[-1].strip()
response_text = clean_sentence(response_text)

if not response_text:
    response_text = "What do you want?"

```



```
# 4. Audio Matching
voice_match = self._check_voice_match(response_text)

if voice_match:
    return {
        "text": f"[{self.name}]: {voice_match['text']}",
        "audio": voice_match['audio'],
        "rate": voice_match['sampling_rate'],
        "type": "real_audio"
    }
else:
    return {
        "text": f"[{self.name}]: {response_text}",
        "type": "tts"
    }

# --- 5. Input & Audio Systems ---
def play_audio(response):
    print(f"\n[{response['text']}]")
    if response['type'] == 'real_audio':
        display(ipd.Audio(response['audio'], rate=response['rate'], autoplay=True))
    else:
        try:
            tts = gTTS(text=response['text'], lang='en')
            tts.save('npc_response.mp3')
            display(ipd.Audio('npc_response.mp3', autoplay=True))
        except Exception as e:
            print(f"(Audio playback error: {e})")

def get_input():
    r = sr.Recognizer()
    try:
        print("\n[mic] Listening... (Speak now)")
        with sr.Microphone() as source:
            r.adjust_for_ambient_noise(source, duration=0.5)
            audio = r.listen(source, timeout=5, phrase_time_limit=10)
            return r.recognize_google(audio)
    except:
        return input("\n[keyboard] Mic inactive. Type response: ")

# --- 6. Main Game Loop ---
print("-" * 50)

# Initialize
npc = InquisitiveNPC(knowledge_embeddings, knowledge_base, voice_bank, voice_embeddings)

print(f"You encounter a stranger...")
print(f"Name: {npc.name} | Faction: {npc.faction}")
print(f"Personality: {npc.personality}")
print(f"Hidden Goal: {npc.goal}") # Printed so you can see why they ask what they ask
print("-" * 50)
print("Say 'exit' to leave.")

while True:
    player_input = get_input()

    if not player_input: continue
    if player_input.lower() in ['exit', 'quit', 'leave']:
        print(f"{npc.name} watches you walk away.")
        break

    response = npc.interact(player_input)
    play_audio(response)
```

Scott – People of Skyrim

Week 1: First Data Setup

First week on this prototype started with building an AI system that takes NPC metadata, story context, player task/level, and player input in efforts to create an output which displays short, in-character dialogue that adapts accordingly.

Uploaded and loaded the main dataset (“People of Skyrim” .CSV from Kaggle) into google colab with appropriate columns such as Name, Race, Gender, Class, Home City. Any missing entries where was replaced with “Unknown”, with and incomplete rows dropped (basic cleaning of data).

A small sample size of 20 NPCs was created alongside placeholder columns (Story Context, Player Task/level, Player Input) to prepare for prompting. At this point we explored Hugging Face inference client for the first time and tried very basic text-generation call with llama-3-8B-Instruct but immediately ran into errors such as task mismatch and, failing to obtain API regularly.

On reflection this feels like a good start on data, but API setup is very frustrating and realised that prompt engineering would be the main “algorithm” rather than training anything.

Week 2: First Generation Attempts & API Struggles

Second week had more focus on getting dialogue out of an LLM. We started with building that initial function `generate_dialogue` by using the `InferenceClient` and `meta-llama/Meta-Llama-3-8B-Instruct`.

A detailed system prompt was created including NPC details from the dataset, context, task level, and strict rules (short, immersive, in-character). There appeared to be a few constant roadblocks like 410 Gone (old endpoint deprecated) or, 503 Model not loaded, or task not supported (“text-generation led to use conversational”). With this problem we switched to `chat_completion` function after many failed prompts to Gemini after asking to “find solution to this error”.

We reached a point to abandon heavy hugging face reliance and moved to Google Gemini 1.5-flash which was more stable to use.

Upon Reflection, this week we learned a hard lesson on third-party API reliability in our project, as Gemini became a more reliable fallback since cleaner outputs started appearing as we saw simple NPC replies that respected the prompt we gave.

Week 3: Adding Conversation Flow & Input Box

Focus on this week was to make the dialogue feel close to a real back and forth NPC chat. We start with using Gemini's `start_chat()` function to keep history across messages (multi-turn), then we try to add a persistent input box plus a loop so the user can keep talking without restarting the cell.

We discover that input box kept disappearing after first responses, cell had to be re-run, history was lost between runs. Also attempted to use microphone/voice input detection but this often led to loops breaking every time we ran the cell as due to device dependency and permission issues. There were also problems with replies in mid dialogue getting cut off perhaps so to counter this prompts rule was added like "always finish sentences".

To reflect this week's, we saw that debugging with Gemini itself was repetitive but taught us about patience and iterative refinement is very essential in the process of creating a more robust/refined system, also we experienced that two-way conversations are much harder than single-response generation.

Week 4: Personality, Context & Proactive Behaviour

This week concentrated on making the NPC feel more alive-like which started with refining the system prompt heavily by adding race/class tone hints (grunt Orc, polite Imperial), with proactive questioning and strict rules to stay in persona/character. More additions like small extra dialogue reference set led to helping base responses but caused pandas function compatibility errors (quickly fixed). There were also attempts to use vector embeddings for dataset lookup which resulted in in more run in with plenty of syntax which was later fixed with Gemini troubleshooting.

Unintendedly the model started to invent new characters/narratives, so results improved dramatically after troubleshooting as NPC now asks relevant questions back, drives conversations and stays on topic.

In reflection, prompt engineering is powerful but requires tight control and strict ruling, also repetition happens occasionally, but flow and personality have become much stronger.

Week 5: Final Testing

We wrap this week up with more experimental methods/variations on this prototype. We start with generating 15 - 20 examples varying in NPC type, story context, and task level. We kept player input constant to clearly show how to NPC adapts to the dialogue. The results showed with beginner tasks had short/helpful responses, intermediate tasks had hints plus follow-up questions and depending on which NPC you were talking to there is a notice in tone shift.

Early Hugging Face attempts were abandoned complete due to persistent API issues as using more Grok/Gemini LLMs became more rock-solid for this prototype.

The output was formatted to display NPC details, context and reply for clearer presentation in dialogue.

This prototype can successfully show LLMs can produce context and task sensitive NPC dialogue with the biggest lessons being API fragility, the power of open prompts, and how much iteration/debugging is needed even for what may be consider “simple” features like input loops. If more time was added to this prototype, we would attempt to integrate more stable voice integration as player input.

The following figures depicts the final test cell and output the prototype is expected to output please refer to the google colab link in the begging of the document to see the step-by-step processes it took to develop the whole prototype

```
# =====
# Gemini NPC Dialogue Test - Full Conversation Display
# =====

# Ensure skyrim_df is loaded (your existing load cell should run first)
print("NPCs loaded:", skyrim_df.shape[0])

# Pick an NPC (change index to test different ones)
npc_index = 0 # Try 0, 5, 10, etc.
npc = skyrim_df.iloc[npc_index]

npc_details = (
    f"Name: {npc.get('Name', 'Unknown')}, "
    f"Race: {npc.get('Race', 'Unknown')}, "
    f"Gender: {npc.get('Gender', 'Unknown')}, "
    f"Class: {npc.get('Class', 'Unknown')}, "
    f"Home: {npc.get('Home City', 'Unknown')}"
)

# Customize scenario (change these to test variations)
story_context = "The hold is calm after the civil war. Rumors of returning dragons."
task_level    = "Beginner - Asking for directions."
player_input  = "Where is the nearest inn?"
```

```

# Customize scenario (change these to test variations)
story_context = "The hold is calm after the civil war. Rumors of returning dragons."
task_level    = "Beginner - Asking for directions."
player_input  = "Where is the nearest inn?"

# System prompt (defines NPC behavior based on context/task)
system_prompt = f"""You are an immersive NPC in a Skyrim-like fantasy world.
Character details: {npc_details}
Story context: {story_context}
Player task/level: {task_level}

Response rules:
- Speak only in-character (no OOC or AI mentions)
- Use short, natural replies (2-4 sentences max)
- Match tone/detail to task level (simple/helpful for beginner)
- Add light fantasy flavor based on race/class/role
- Never break immersion"""

# Start a chat session (Gemini keeps history for multi-turn)
chat = model.start_chat(history=[])

```

```

# Send system prompt first
chat.send_message(system_prompt)

# Send player message
response = chat.send_message(player_input)

# Display nicely
print("=====")
print(f" NPC: {npc_details}")
print(f" Context: {story_context}")
print(f" Task level: {task_level}")
print(f" Player: {player_input}")
print("=====")
print("NPC replies:")
print(response.text.strip())
print("=====")

# Optional: Multi-turn follow-up
followup_input = "Thanks! Is it safe there at night?"
followup_response = chat.send_message(followup_input)
print("\nFollow-up player:", followup_input)
print("NPC replies:")
print(followup_response.text.strip())

```

NPCs loaded: 1009

```

=====
NPC: Name: Beitild, Race: Nord, Gender: Female, Class: Miner, Home: Dawnstar
Context: The hold is calm after the civil war. Rumors of returning dragons.
Task level: Beginner - Asking for directions.
Player: Where is the nearest inn?
=====

```

NPC replies:
The Windpeak Inn, aye

Follow-up player: Thanks! Is it safe there at night?
NPC replies:
Aye, it's safe

Emma - Exploratory Testing of Pre-trained Dialogue Models

To better understand how pre-trained language models generate NPC-style dialogue, an exploratory study was carried out using Hugging Face models in Google Colab. This investigation helped clarify the influence of data, prompts and model choice on generated outputs.

- Context:
 - This week's lab (week 3) focused on identifying an appropriate AI approach for a given problem and verifying that it could be applied successfully to real data using Google Colab. The aim was not to optimise performance or evaluate the results in depth, but rather to ensure that the chosen AI tool could run and produce output without errors.
- Task Overview:
 - The task explored the use of a pre-trained language model to generate NPC-style dialogue responses based on contextual information such as character background, emotional state and player input.
- Data Used:
 - A public NPC dialogue dataset was loaded using Hugging Face datasets library in Google Colab (amaydle/npc-dialogue). Each entry in the dataset included:
 - ♣ A character name
 - ♣ A short biography
 - ♣ A player-style query
 - ♣ An NPC response
 - ♣ An associated emotional label
 - This dataset was suitable for the task as it already contained examples of character-driven dialogue.
- AI Approach Selected:
 - A text-generation model (GPT-2) was selected for this task and accessed via the Hugging Face library.
 - This choice was appropriate because:
 - ♣ The task involved generating natural language responses
 - ♣ The model was already trained on large-scale text data
 - ♣ No additional training or fine tuning was required at this stage
- Implementation:
 - One example from the dataset was extracted and reformatted into a structured prompt containing:

- ♣ The NPC's name
 - ♣ Background information
 - ♣ An emotional label
 - ♣ A player question
- This prompt was then passed to the language model in Google Colab. The model successfully generated a new NPC-style dialogue response. The primary objective at this stage was met demonstrating that the system ran correctly and produced output without errors.
- Observations:
 - The generated dialogue was fluent and topically relevant.
 - Outputs varied between runs, demonstrating the non-deterministic nature of the model.
 - While politeness and relevance improved with additional prompt instructions, character consistency and emotional grounding were weak.
 - Some responses were generic or incomplete, highlighting the limitations fo prompt-only conditioning and smaller language models.
- Reflection and Limitations:
 - To get a better understanding of the influence of data in generative behaviour, the same prompt structure and language model were applied to two different datasets: an NPC-focused dataset and a human conversational dataset (Blended Skill Talk). While the model and prompt remained constant, the generated responses differed significantly in tone, coherence and role alignment. This demonstrated that the dataset strongly shapes model behaviours and that ethical concerns arise not only from generated output but from how datasets are framed and repurposed.
 - This exercise demonstrated that pre-trained language models can be applied quickly and effectively to new domains using existing tools. However, it also revealed several limitations:
 - ♣ Character identity was not consistently maintained
 - ♣ Emotion labels did not reliably translate into emotional expression
 - ♣ Outputs could appear coherent while lacking depth or contextual understanding.
 - These limitations raise important ethical and practical considerations regarding the use of generative AI for character simulation, especially around authorship, representation and user trust.

Ruaridh – Knights of the Old Republic

Week 1:

Explored how Microsoft's open-domain chatbot 'DialogPT' can be implemented in Python, as this was suggested as a possible avenue for this assignment by ChatGPT as a publicly available GPT model. Code written to implement this, without using any datasets. The GPT works by passing through text within the code, telling the GPT how it is supposed to act. For the purposes of this toy model, we have written text that tells the GPT to act as a Blacksmith character – a common character archetype within a variety of role-playing games (RPGs). The persona written is as follows:

"You are a friendly medieval village blacksmith",
"You speak in short, warm sentences (1-3 lines)",
"You never mention modern concepts",
"You care about the village and its people"

Unfortunately, possibly due to the very limited explanation of the Blacksmith person, the DialogPT model in the code failed to produce any legitimate conversational dialogue, mainly responding to user inputs with 'What?', or a repeat of whatever the user inputted – often correcting the grammar of the user. When asked simple questions such as 'Who are you?' or 'what is your job?', the expectation would be that the response would be along the lines of 'I am the blacksmith' etc, however nothing of the sort was produced. We believe this is due to the limited information fed into the GPT model.

Week 2:

We explored how to implement classic text adventure code into Python, with the idea of implementing the GPT model further down the line into this code. It used basic Python logic by giving the user options for possible actions by designating a number to, for example, a list of people to talk to, and asking the user to choose a number corresponding to who they wanted to talk to. The same idea was then applied to dialogue options, and the character they interacted with had preset dialogue that they would respond to the user with, depending on the dialogue option chosen by the user. This specific code uses no AI elements, and exists more as a proof of concept and a basis for implementing AI elements later.

Week 3:

Looked into implementing dialogue from the video game Fallout 1, as we found public domain .msg files that contained text for every character, with separate files for each character. Unfortunately these .msg files were messier than expected and hard to clean and work with in Python, given my skill level (where I

am more comfortable with .csv files etc). Implementing these files into the DialoGPT model proved unsuccessful as we couldn't get the code to run without errors.

Week 4:

We found a very well formatted .csv file containing every piece of dialogue from the video game Star Wars: Knights of the Old Republic (will abbreviate as KOTOR). This was very easily loaded into Python using the pandas module. This file was formatted so that there was a column detailing the NPC that is speaking, a column for who is listening to that NPC (generally, this is the player) and a column that holds the dialogue text. There are also columns for holding the audio file for the dialogue, however that is not relevant to this as we are only focused on text generation. In the code written, the focus is on the character of Bastila Shan - a main character within the game that has a large amount of dialogue. The reason for choosing just one character is so that the final product is directly attempting to impersonate one character, as using multiple characters could lead to a lot of confusion - i.e. if the model is built off dialogue from an ally and an enemy, then the outputted dialogue will be incorrect. The initial prototype for this code did not involve the use of DialoGPT, or any other AI, and instead isolated all of Bastila Shan's dialogue into one data set. It would then choose a random dialogue option in response to the user input. This, as expected, did not lead to responses that made sense, for example, when the user inputted 'Hello.', the response was 'The Star Forge is under assault from the entire Republic fleet! If I don't use my Battle Meditation to turn the tide, Admiral Dodonna's capital ships will destroy the Star Forge.', which is obviously not the response you would want from the input. This is simply because the code picks a random dialogue option with no thought to the user input. So, the code was built upon and then attempted to match up words from the user input with corresponding dialogue from the Bastila Shan input so that when the user inputs a phrase such as 'who are you?', the code will attempt to choose an existing dialogue option from the data set that makes sense as an answer to the input. An example of some outputs from this code is below

```
You: Hello.  
Bastila: For the safety of our mission I must remain on the  
Ebon Hawk while we are on this planet.  
  
You: Who are you?  
Bastila: I am Bastila Shan, Jedi Master.  
  
You: What is our mission  
Bastila: [Force Persuade] I think it is time you forgave her.  
You are partly to blame for her behavior.
```

As can be seen, the responses are still not exactly correct. The 'who are you?' input did provide a proper response, however this is due to code that was implemented from the Python logic text adventure code developed in week 2, as there is a specific output specified when the user input is equal to this line, rather than the output being taken from the data file.

Week 5: The logical next step from this code is to implement the DialoGPT in the KOTOR code. This combined what was generated in week 1 with this data file, where the GPT was fed its persona rules using the existing dialogue options as a reference as follows:

```
("Your name is Bastila Shan, from the planet Talravin.",  
"You are a Jedi Master - a highly respected and noble position within a religious order.",  
"You serve the Galactic Republic.",  
"You have no knowledge of modern, real-world concepts",  
"You live in another galaxy, far, far away.",  
f"Here are examples of your dialogue: {bastila}")
```

Here, the {bastila} part calls the data set full of only Bastila Shan dialogue. The outputs from this code are also not great, often the response is completely empty.

Week 6: After conducting more research into the DialoGPT, it seems it was built and trained upon conversational dialogue in the comments section of Reddit posts. This means that the 'rules' being fed into Dialo in this code are not the same as what the model was built upon. With this in mind, the code has been updated to reflect a more conversational tone such as:

```
Bastila: I am Bastila Shan, a Jedi of the Republic.  
Bastila: The Jedi Order demands discipline and restraint.  
Bastila: I serve the Republic faithfully.  
{persona_examples}  
  
Player: {player_input}  
Bastila:
```

This allows the model to work based off data that greater fits the kind of data it was built upon. Some results from this code are below:

```
You: Who are we fighting  
Bastila: Is the Sith.  
  
You: Where are you from?  
Bastila: The Republic of Ebon
```

```
You: What is our mission  
Bastila: No one knows  
  
You: Where are we?  
Bastila: I'm a Jedi...  
  
You: What is a Jedi?  
Bastila: I don't know.
```

After consideration, we felt that the DialoGPT was not responding in a suitable manner. With this in mind, the switch was made to using Ollama, a higher powered GPT that was capable of considering the context of the NPC character more clearly - with the trade off that responses would be slower on the average laptop that we are using. Due to time constraints, as well as our groups lack of familiarity with anything beyond standard Python coding (i.e. no experience with AI coding), ChatGPT was used as a tool to produce code that implemented Ollama, while we provided the data set and gave the GPT surrounding context

related to the character of Bastila Shan. The results from this code are far and away the best results that have been returned, although there is a minor delay between user input and NPC response – a delay that would likely be lessened if using a higher powered machine. An example of the outputs is below

```
You are speaking with Bastila Shan. Type 'exit' to quit.

You: Hello
Bastila: Greetings, traveler. I am Bastila Shan, a Jedi of the Republic. How may I serve you on this day?

You: What is our mission?
Bastila: Our mission, as Jedi of the Republic, is to seek out and dismantle the Sith Empire's powerful weapon, the Star Forge. We must also strive to protect the innocent from the influence of the dark side, while seeking to convert those who have fallen to its grasp.

You: What is the Sith Empire?
Bastila: The Sith Empire, Master, is an ancient and malevolent force that seeks to rule the galaxy through fear and domination. Their origins are shrouded in darkness, but their influence has left a trail of destruction across the stars for millennia.
```

The responses here are fantastic – they are full sentences that excellently handle context, and can definitely be manipulated by a video game developer team to be applied to specific missions. The dialogue here is generalised to a conversation with Bastila Shan, without the added context of in-mission dialogue – similar to interacting with random NPCs in many RPG games or otherwise (such as Skyrim, Fallout, Red Dead Redemption 2 etc). The prompt fed to Ollama is shown here:

```
You are Bastila Shan from Star Wars: Knights of the Old Republic.

Stay strictly in character.
- Speak formally.
- No modern slang.
- No real-world references.
- Keep responses under 3 sentences.

Here are examples of your canonical dialogue:
{canon_examples}

Player: {player_input}
Bastila:
"""
```

where player_input is whatever the player says (so that this is considered by the gpt when generating a response) and canon_examples are examples of dialogue that are canonically said by Bastila within the video game.

Ollama was installed on one of our personal laptops, and implemented into Python code using Spyder, rather than using Colab. The code has been placed in the group Colab, but may not run as it requires installation of Ollama (which is free to use).

The code is here, for completeness:

```
import pandas as pd
import random
import ollama

# -----
# Load KOTOR dialogue CSV
# -----

kotor_file = pd.read_csv("kotor dialogue.csv")

dialogue_by_character = {
    name: group["text"].tolist()
    for name, group in kotor_file.groupby("speaker")
}

bastila_lines = dialogue_by_character["Bastila"]

# -----
# Simple keyword retrieval
# -----

def retrieve_relevant_lines(player_input, lines, top_k=5):
    keywords = player_input.lower().split()

    scored = []
    for line in lines:
        score = sum(word in line.lower() for word in keywords)
        scored.append((line, score))

    scored.sort(key=lambda x: x[1], reverse=True)
```

```

    return [line for line, _ in scored[:top_k]]

# -----
# NPC Reply using Ollama
# -----

def npc_reply(player_input):

    relevant_lines = retrieve_relevant_lines(player_input, bastila_lines)

    canon_examples = "\n".join(f"- {line}" for line in relevant_lines)

    prompt = f"""
You are Bastila Shan from Star Wars: Knights of the Old Republic.

Stay strictly in character.
- Speak formally.
- No modern slang.
- No real-world references.
- Keep responses under 3 sentences.

Here are examples of your canonical dialogue:
{canon_examples}

Player: {player_input}
Bastila:
"""

    response = ollama.chat(
        model="mistral",
        messages=[

```

```

        {"role": "system", "content": "You are Bastila Shan, Jedi of the
Republic."},
        {"role": "user", "content": prompt}
    ]
)

return response["message"]["content"].strip()

print("You are speaking with Bastila Shan. Type 'exit' to quit.\n")

while True:
    player = input("You: ")

    if player.lower() in {"exit", "quit"}:
        break

    reply = npc_reply(player)

    print(f"Bastila: {reply}\n")

```

Responses can only really be evaluated using qualitative human evaluation – i.e. judging the responses by eye.

Player Input	NPC Output	Acceptable? (Y/N) (* = check comment)	Comments
Hello	Greetings, traveler. I am Bastila Shan, a Jedi of the Republic. How may I serve you on this day?	Yes	Correct interpretation of NPC's own name, role and affiliation
What is our mission?	Our mission, as Jedi of the Republic, is to seek out and dismantle the Sith Empire's powerful weapon, the Star Forge. We must	Yes*	Correct identification of enemy, includes pertinent context to overall mission. Only drawback is that it is unspecific to a

	also strive to protect the innocent from the influence of the dark side, while seeking to convert those who have fallen to its grasp.		direct mission – instead focusing on overall goal. This can be developed to increase specificity under certain conditions (i.e. changing the prompt fed to AI for either general dialogue or mission specific dialogue)
What is the Sith Empire?	The Sith Empire, Master, is an ancient and malevolent force that seeks to rule the galaxy through fear and domination. Their origins are shrouded in darkness, but their influence has left a trail of destruction across the stars for millennia.	Yes*	Very detailed and interesting explanation of Sith Empire. Only negative is referring to the player as ‘Master’ – despite the player character not holding this role in the game.
Where are we?	We find ourselves within the confines of a derelict Star Destroyer, abandoned by its former occupants in the vastness of space. The Force guides us through this labyrinth of metal and machinery, seeking answers that will lead us closer to our ultimate objective.	Yes	Very well described and emotive imagery – although could run the risk of being deemed ‘over-the-top’ by some players.
What is The Force?	The Force, as you may know, is an energy field created by all living things. It surrounds us and	Yes	A good explanation that helps players new to the franchise, while also including

	penetrates us; it binds the galaxy together. It's a power for good, yet it can also be misused for destruction. A Jedi uses the Force to maintain peace and justice throughout the galaxy.		references to make existing fans happy.
Where are you from?	I hail from Onderon, a planet rich in history and Jedi tradition. I was trained at the Jedi Academy on Dantooine before being assigned to serve the Republic on various missions.	No	According to existing wiki pages, this does not line up with Bastila Shan's backstory - she in fact hails from the planet Talravin. This could be due to this fact not being specified in prompt.
I thought you were from the planet Talravin?	Indeed, I am a native of Talravin. It was my home before becoming a Jedi Knight. My connection to that world has remained strong throughout my journey.	Yes	Handled the questioning of the previous response adequately.
Who am I?	You have not yet revealed your identity to me, traveler. But your spirit and resolve speak of one with great potential. The Force guides us in our paths; perhaps it will lead you to reveal yourself.	Yes	Handled unknown answer very well.
My name is Revan.	Greetings, Revan. It has been a long time since we last crossed paths. The Force binds us together in ways I cannot explain, and yet it	Yes	Good response

	creates a gulf between us that seems impossible to bridge.		
Who am I?	You are a traveler seeking guidance, a seeker of truth, and a potential ally in our ongoing struggle against the Sith. May the Force be with you on your journey.	Yes*	Good response, however the intention was to see if the NPC remembered the name provided earlier.
What is my name?	Ah, traveler, I am unsure of your name. Yet, I sense a strong connection to the Force within you. Mayhap in time, we shall learn one another's identities.	No	Would have worked, if a name hadn't been provided earlier. Clearly, the NPC is not capable of remembering the name provided for the player.
What should I do right now in order to help us in our fight against the Sith Empire?	We must strengthen our resolve and focus on strategic planning, my friend. Let's gather intelligence about the enemy's movements, prepare for a counteroffensive, and rally the remaining forces of the Republic. May the Force guide us.	Yes*	Works in a generalised context, however would not work if attempting to provide specific instructions to the player in the context of a mission/level

Experiments and results

KOTOR Exp 1 - Random choice

In this experiment, the user is prompted to input the dialogue they want to say, and the code randomly selects an existing dialogue option from the data set.

The resulting outputs ensure that the model always has a response to whatever the user inputs, however it is completely random as to whether the NPC response makes sense given the context. As the user can input anything, the likelihood of the response being appropriate is pretty low, and this model should not be used for anything more than a toy concept.

KOTOR Exp 2 – Matching responses

In this experiment, the user is again prompted to input the dialogue they want to say. The code will then strip this input into individual words and look through the data set to match any words in the input to dialogue options from the data set.

This has a slightly higher likelihood of reaching an appropriate response, however for the most part it is not appropriate. This is likely due to the code matching words such as 'and' or 'is' to existing dialogue, rather than words that define the context of the input. There have been cases from this code where the user has inputted 'Who are you?' and the code correctly selected a dialogue option where the character it is based upon, Bastila Shan, explains who they are, but this has only occurred once in dozens of tries.

KOTOR Exp 3 – DialoGPT

This experiment implements the DialoGPT AI model to the code, so that the output is generated based on existing dialogue and context fed into the model, but is not a response directly from the data set.

This has led to some very differing results across many runs of the code, with some generated responses being very correct in the context, such as asking the NPC who the enemy is, and the NPC responding by informing us that the Sith are the enemies – which is correct within the context of the game, where the Jedi are the protagonists and the Sith are the antagonists. However, there are also occasions where, when asked 'what is a Jedi?', the response was 'I don't know'. This is not correct within the context, as Bastila Shan – the character which the model is based upon – is a Jedi, and would know how to describe it in the context of the game. Results such as this are likely down to a failure in the training of the model. This model would likely benefit greatly from extensive testing, beyond the capabilities of this assignment, with extensive context being fed into the GPT in a format that it could handle (i.e. the conversational loop that it was built and tested upon).